ideis · Uniwersytet DSW

Architektura Aplikacji w Pythonie

[Lab 4] Python i bazy danych

Michał Madejski

Laboratoria · 25 kwietnia 2026

Co umiesz po tych zajęciach?

Trzy konkretne umiejętności, które zabierzesz do domu:

01

Zapiszesz dane z Pythona do bazy

PostgreSQL + psycopg, MongoDB + PyMongo. CRUD, transakcje, context manager — tak, jak to robi się w prawdziwych projektach.

02

Rozróżnisz typy baz

Relacyjna vs. dokumentowa vs. szeregów czasowych vs. wektorowa. Zrozumiesz, kiedy wybrać co — i dlaczego to ma znaczenie w 2026.

03

Zobaczysz, po co bazy wektorowe

ChatGPT i RAG zmieniły świat baz. pgvector, Qdrant, Pinecone — poznasz, co się wydarzyło i dlaczego każdy o tym teraz mówi.

Na końcu zajęć wracamy do tej listy — sprawdzimy, czy wszystko zaskoczyło.

Agenda · 90 minut

0:00 – 0:10	**Wstęp**	po co w ogóle bazy · historia · jak wybierać
0:10 – 0:25	**Przekrój baz 2026**	relacyjne · dokumentowe · key-value · szeregi · wektory
0:25 – 0:50	**PostgreSQL + Python**	psycopg · CRUD · transakcje · DEMO Titanic
0:55 – 1:05	**MongoDB + Python**	PyMongo · filtracja · agregacja · DEMO filmy
1:05 – 1:20	**Bazy specjalistyczne**	szeregi czasowe · wektory · pgvector · kiedy co
1:20 – 1:30	**Zadania + quiz**	samodzielnie · retrieval check

Dlaczego bazy danych?

Oracle · 1977

Larry Ellison buduje pierwszy komercyjny SQL na podstawie pracy Edgara Codda z IBM. Pół wieku później Oracle wciąż jest #1 na liście DB-Engines — a rynek baz urósł do setek systemów dla konkretnych zastosowań.

Jedna baza dla wszystkiego to dziś antywzorzec. Dobra baza = dobrana do danych.

Baza danych robi trzy rzeczy, których plik CSV nie zrobi:

Szuka szybko

Index pozwala znaleźć 1 wiersz w 100 mln w milisekundach — nie sekundach.

Chroni spójność

Transakcje (ACID) gwarantują: albo wszystko, albo nic. Przelew nigdy nie znika w połowie.

Obsługuje wielu naraz

1000 użytkowników zapisuje do tabeli jednocześnie i żaden nic nie psuje.

W Pythonie standard: psycopg (PostgreSQL) + pymongo (MongoDB) + pgvector (wektory) — wszystko przez PEP 249.

Relacyjna vs. nierelacyjna

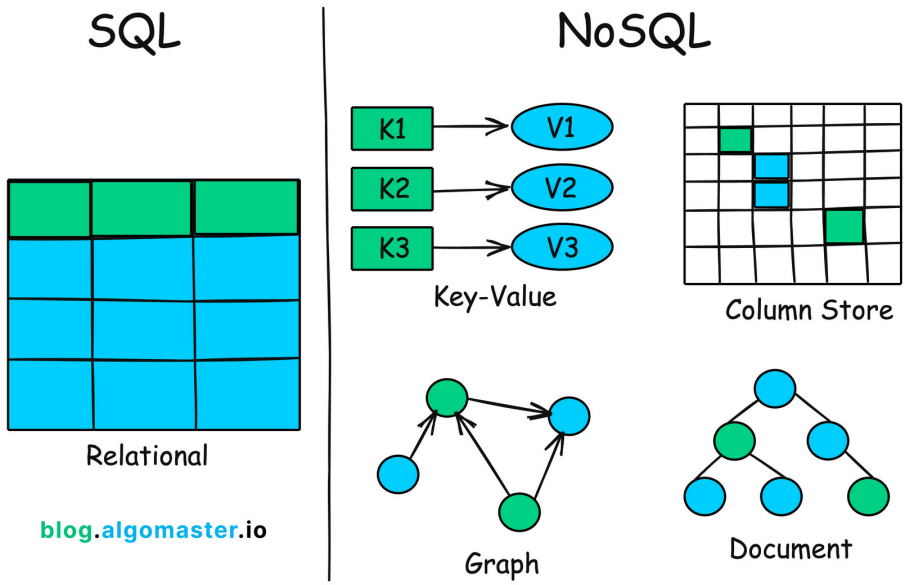
Relacyjna (SQL)

Dane w tabelach · wiersze + kolumny · sztywny schemat · SQL

- Zdefiniowany schemat — kolumny mają typy
- Relacje między tabelami (foreign keys)
- Transakcje ACID — spójność > wydajność
- Język SQL — jeden standard od 40 lat
- Idealne: e-commerce, księgowość, CRM

Przykłady:

PostgreSQL · MySQL · Oracle · SQL Server · SQLite



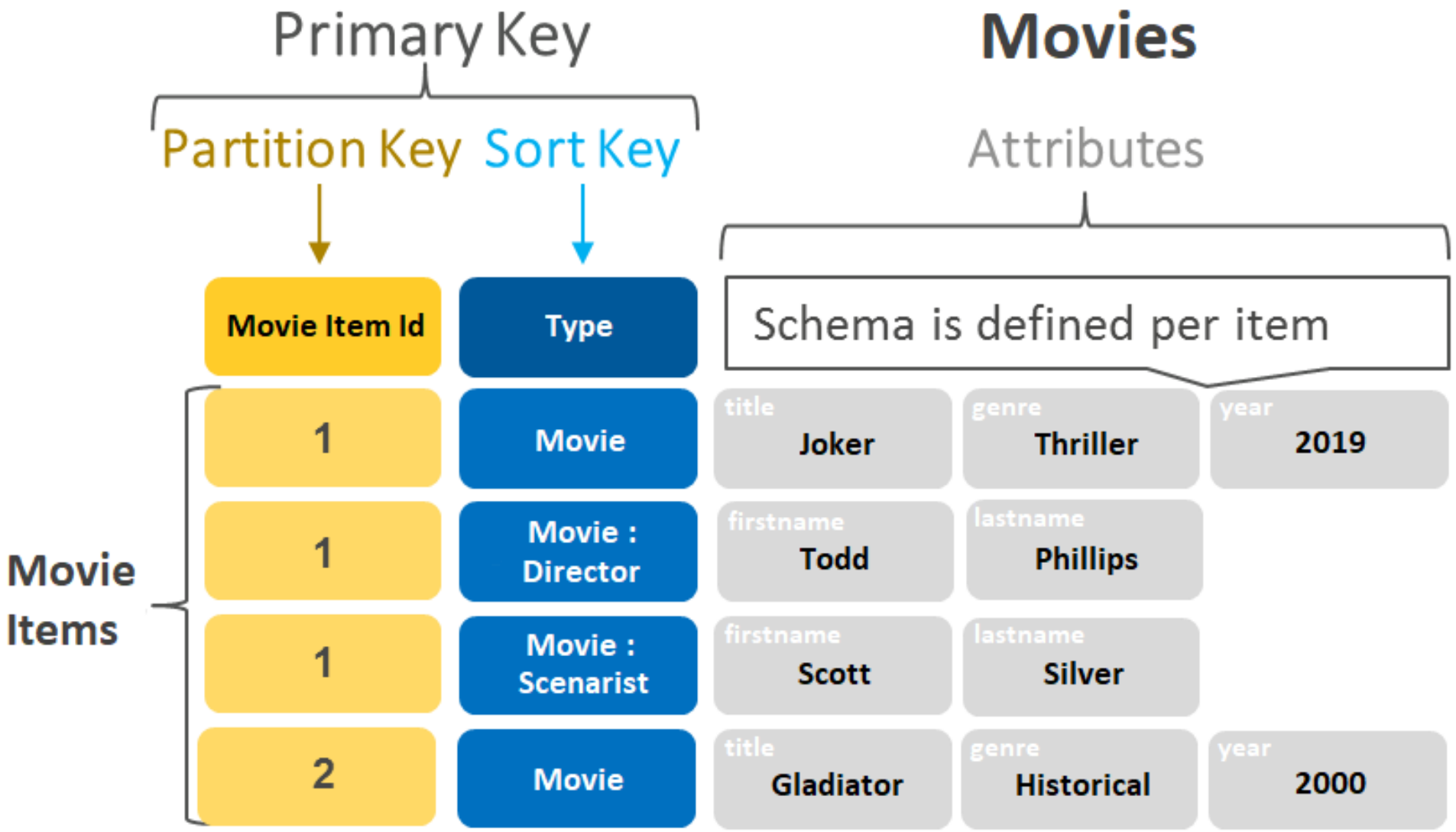
Nierelacyjna (NoSQL)

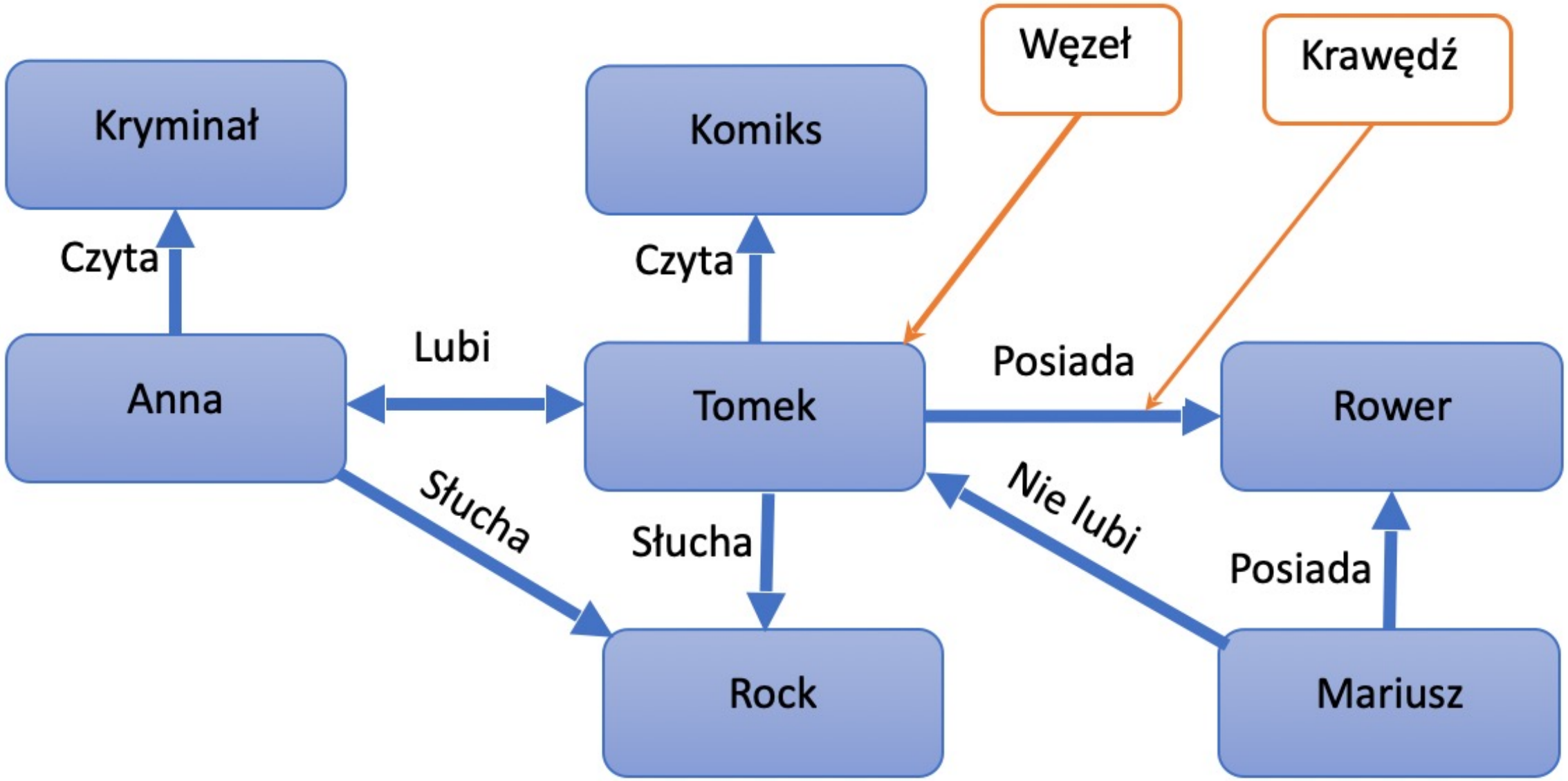
Dokumenty · klucz-wartość · grafy · kolumny · elastyczność

- Schema-less lub bardzo elastyczny schemat
- Skalowanie poziome (sharding out-of-the-box)
- Struktury zagnieżdżone (JSON-like)
- Brak jednego standardu — każda ma swoje API
- Idealne: logi, real-time, dane półstruktur.

Przykłady:

MongoDB · Redis · Cassandra · Neo4j · DynamoDB


Primary Key
Movies
Partition Key
Sort Key
Attributes
Movie Item Id
Type
Schema is defined per item
Movie Items
1
Movie
title
Joker
genre
Thriller
year
2019
1
Movie : Director
firstname
Todd
lastname
Phillips
1
Movie : Scenarist
firstname
Scott
lastname
Silver
2
Movie
title
Gladiator
genre
Historical
year
2000


Węzeł
Krawędź
Kryminał
Komiks
Czyta
Czyta
Lubi
Anna
Tomek
Posiada
Rower
Słucha
Słucha
Nie lubi
Posiada
Rock
Mariusz

Pięć typów baz · do czego służą

Każdy typ jest dobry do innych danych. Dopasowujesz bazę do problemu, nie odwrotnie.

	Przykłady	Kiedy używać
Relacyjna Tabele, wiersze, SQL	**PostgreSQL · MySQL**	faktury, zamówienia, księgowość
Dokumentowa JSON-podobne dokumenty	**MongoDB · CouchDB**	katalogi produktów, CMS, logi
Klucz-wartość Najprostsza: klucz → wartość	**Redis · DynamoDB**	cache, sesje, rate limit
Szeregi czasowe Dane z timestampem	**TimescaleDB · InfluxDB**	metryki IoT, ceny akcji, monitoring
Wektorowa Embeddingi AI (1536-dim)	**pgvector · Qdrant**	semantic search, RAG, ChatGPT

Kto rządzi w 2026? · DB-Engines

Ranking oparty na Google, Stack Overflow, ofertach pracy i LinkedIn. Top 5 nie zmienia się od lat.

#1	Oracle	*Relacyjna*
#2	MySQL	*Relacyjna*
#3	Microsoft SQL Server	*Relacyjna*
#4	**PostgreSQL**	*Relacyjna*
#5	**MongoDB**	*Dokumentowa*
#6	Snowflake	*Analityczna*
#7	Redis	*Key-value*
#8	Databricks	*Analityczna*

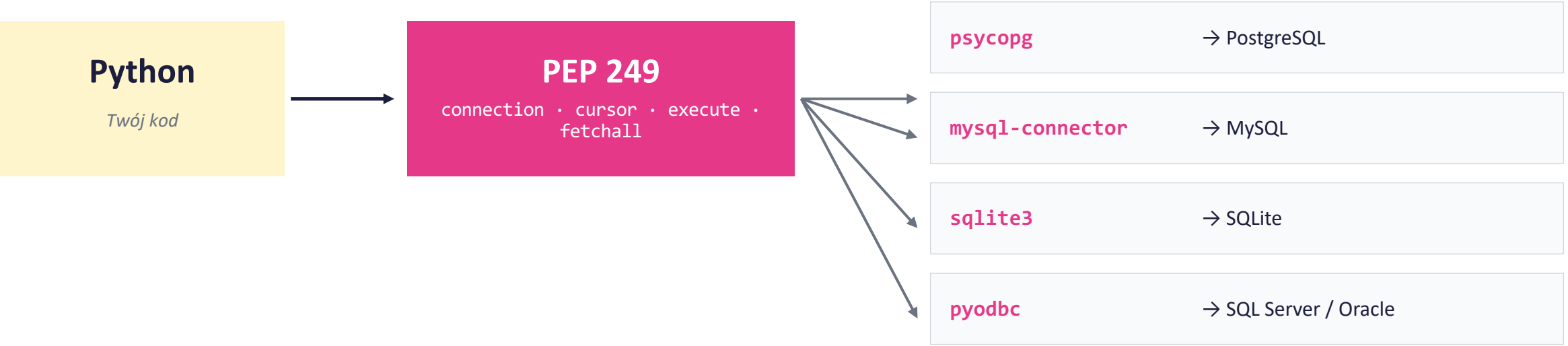
Co to znaczy dla Ciebie?

- 4 z 5 najpopularniejszych baz to SQL. SQL nie umiera — rośnie.
- PostgreSQL urósł najmocniej z top-5 (open source, silna społeczność).
- MongoDB dominuje świat dokumentowy — de facto standard.
- Dlatego dziś uczymy: Postgres + Mongo. Reszta to ich warianty.

Źródło: db-engines.com/ranking (marzec 2026)

PEP 249 · jeden interfejs dla wszystkich

Python Database API Specification v2.0 — dzięki niemu połączysz się z każdą relacyjną bazą w TEN SAM sposób.



Co to daje praktycznie?

Nauczysz się psycopg raz, a napiszesz kod, który po zmianie importu i connection stringu działa z MySQL, SQLite czy Oracle. Zasada: program do interfejsu (connection, cursor), nie do konkretnej biblioteki. To jeden z powodów, dla których Python podbił świat backendu. Na laboratorium pokażemy demo w sqlite3 (działa bez serwera) — kod psycopg jest niemal identyczny, różni się tylko placeholder %s zamiast ?.

PostgreSQL · szwajcarski scyzoryk

Otwarty, relacyjny silnik rozwijany od 1996 roku. Obecnie uważany za najbardziej zaawansowany open-source RDBMS na świecie.

Backend webowy

Instagram, Reddit, Spotify — wszyscy na Postgres

Analityka

window functions, CTE, JSON — SQL lepszy niż w MySQL

Geografia

rozszerzenie PostGIS — standard światowy dla map

AI i wektory

pgvector — pokażemy dalej, to gorący temat 2026

fakty · 2026

30 lat

rozwoju społecznościowego

#4

na DB-Engines (2026)

+8.2

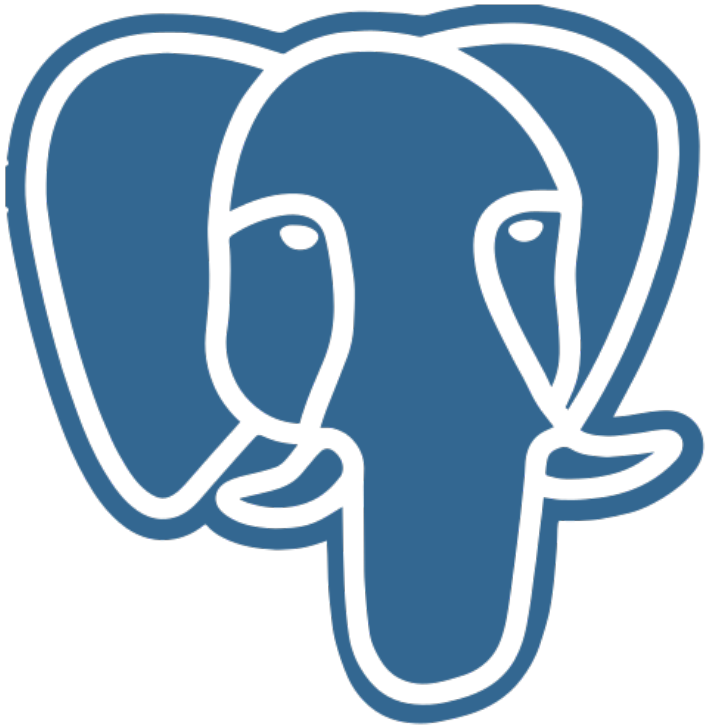
punktów rocznie — rośnie

~100

typów danych wbudowanych

JSON

obsługiwany natywnie



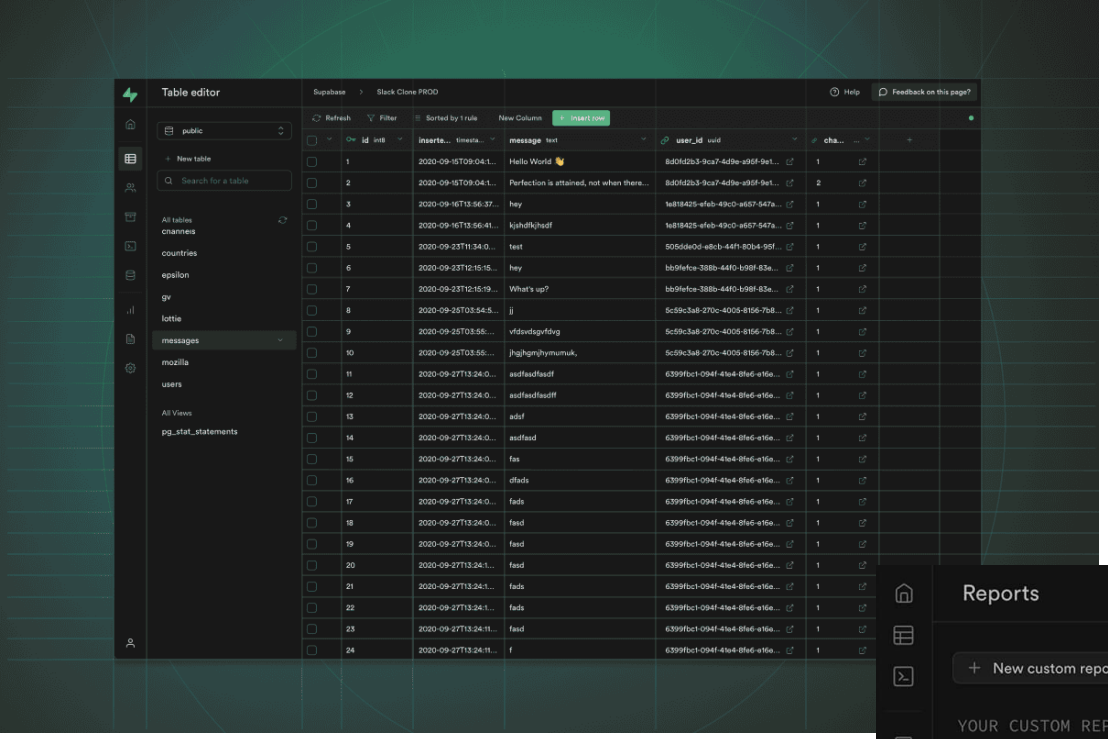
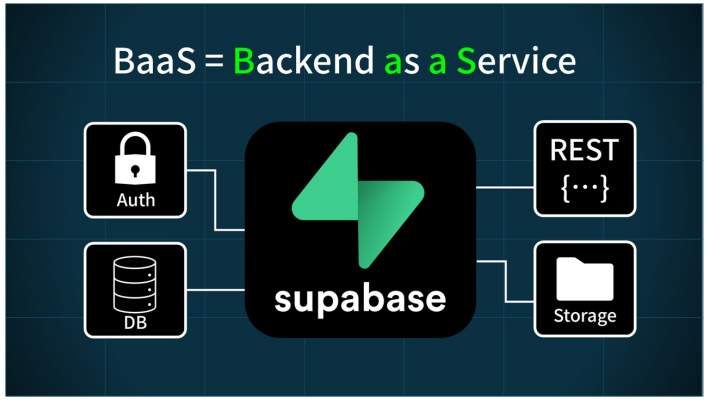
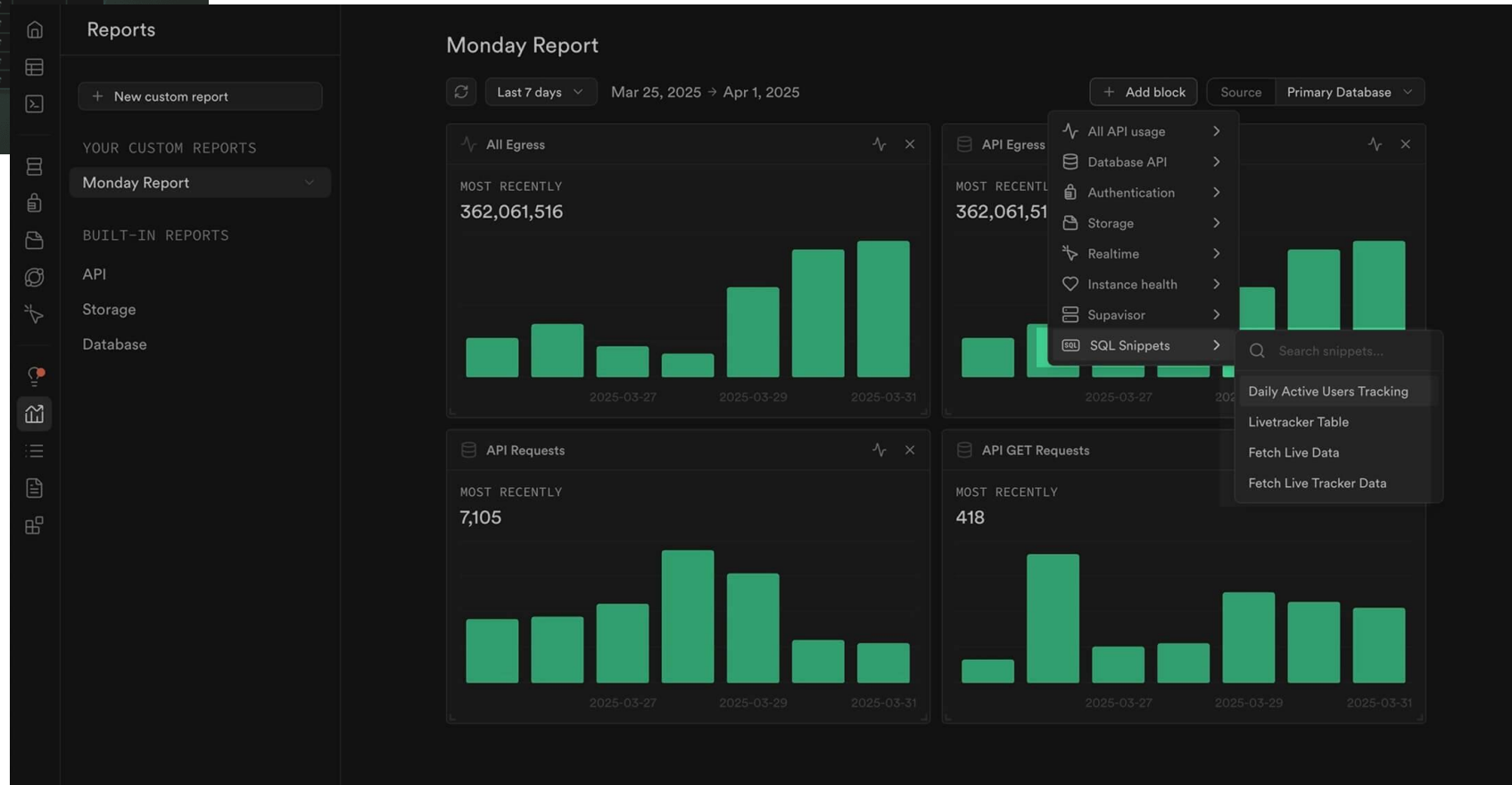


Table editor
Supabase
Slack Clone PROD
Help
Feedback on this page?
public
New table
Search for a table
All tables
channels
countries
epsilon
gv
lottie
messages
mozilla
users
All Views
pg_stat_statements
Refresh
Filter
Sorted by 1 rule
New Column
Insert row
id int8
inserte... timesta...
message text
user_id uuid
cha...
Hello World
Perfection is attained, not when there...
hey
kjshdfkjhsdf
test
hey
What's up?
jj
vfdsvdsgvfdvg
jhgjhgmjhymumuk,
asdfasdfasdf
asdfasdfasdff
adsf
asdfasd
fas
dfads
fads
fasd
fasd
fasd
fads
fads
fasd
f

supabase


BaaS = Backend as a Service
Auth
DB
supabase
REST
{···}
Storage


Reports
New custom report
YOUR CUSTOM REPORTS
Monday Report
BUILT-IN REPORTS
API
Storage
Database
Monday Report
Last 7 days
Mar 25, 2025 → Apr 1, 2025
Add block
Source
Primary Database
All Egress
MOST RECENTLY
362,061,516
2025-03-27
2025-03-29
2025-03-31
API Egress
All API usage
Database API
Authentication
Storage
Realtime
Instance health
Supavisor
SQL Snippets
Search snippets...
Daily Active Users Tracking
Livetracker Table
Fetch Live Data
Fetch Live Tracker Data
API Requests
MOST RECENTLY
7,105
API GET Requests
MOST RECENTLY
418

Instalacja · zależnie od systemu

Jedna z tych trzech komend uruchomi Ci serwer Postgres lokalnie:

Linux / Colab

```
# apt-based (Ubuntu, Debian)
sudo apt-get update
sudo apt-get install postgresql
sudo service postgresql start

# W Colab dopisujesz ! na początku
```

macOS

```
# Homebrew — najwygodniej
brew install postgresql
brew services start postgresql

# Sprawdź że działa:
psql -d postgres
```

Windows

```
# Pobierz instalator z:
# postgresql.org/download

# Dodaj do PATH:
# C:\Program Files\PostgreSQL\18\bin

psql -U postgres
```

Pro-tip · Docker to najczystszy wybór:

```
docker run --name pg -e POSTGRES_PASSWORD=ds -p 5432:5432 -d postgres
```

Użytkownik + baza · minimum startowe

PostgreSQL wymaga konta użytkownika (roli) i bazy. Domyślnie istnieje superużytkownik postgres — przez niego tworzysz resztę.

1

Stwórz użytkownika

```
CREATE USER datascience WITH ENCRYPTED PASSWORD 'datascience';
```

ENCRYPTED = hasło leci do bazy zahashowane, nie jako plaintext.

2

Stwórz bazę i daj mu właścicielstwo

```
CREATE DATABASE datascience WITH OWNER datascience;
```

Jeden user = jedna baza = izolacja. Standard security.

3

Nadaj uprawnienia

```
GRANT ALL PRIVILEGES ON SCHEMA public TO datascience;
```

Od Postgres 15 trzeba dodatkowo — domyślnie public jest restrykcyjny.

Wszystko przez psql lub z -c (tryb skryptowy): sudo -u postgres psql -c "..."

psycopg · łączymy Python z bazą

Najpopularniejsza biblioteka do PostgreSQL w Pythonie. Instalacja: ***pip install psycopg[binary]***

```python
import psycopg

connection = psycopg.connect(
    user="datascience",
    password="datascience",
    host="localhost",
    port=5432,
    dbname="datascience"
)

# sprawdź że działa
with connection.cursor() as cur:
    cur.execute("SELECT version()")
    print(cur.fetchone())
```

4 koncepty

connection

Aktywna sesja z bazą. Jeden connection = jedno gniazdko TCP.

cursor

Narzędzie do wykonywania zapytań. Bufor wyników. Jedna sesja może mieć wiele.

execute

Wysyła SQL do bazy. Wspiera parametryzację %s — używaj ZAWSZE, nie f-stringów (SQL injection).

fetchone/all

Pobiera wyniki. Każdy wiersz to krotka (tuple) lub named tuple.

CRUD · tworzenie tabeli i wstawianie

Tworzymy tabelę

```
create_table = """
CREATE TABLE IF NOT EXISTS Employee (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    salary NUMERIC NOT NULL
)
"""

cur.execute(create_table)
connection.commit()
```

Wstawiamy dane (bezpiecznie)

```
insert = """
INSERT INTO Employee (name, salary)
VALUES (%s, %s)
"""

# %s = placeholder, NIE f-string!
cur.execute(insert, ("Anna", 12000))
connection.commit()
```

Dlaczego %s a nie f-string?

```
# ❌ Naiwnie — SQL injection:
name = "Anna'; DROP TABLE..."
cur.execute(f"INSERT ... '{name}'")
```

```
# ✓ psycopg sam escapuje:
cur.execute(sql, (name,))
# Nawet złośliwy input = bezpieczny
```

commit · bez niego nic się nie zapisze

PostgreSQL owija każdy INSERT / UPDATE / DELETE w transakcję. Dopóki nie zrobisz .commit() — zmiany są tylko "w Twojej głowie".

Python → cursor → SQL → commit() → Baza: ✓ zapisane

rollback() · cofnij zmiany

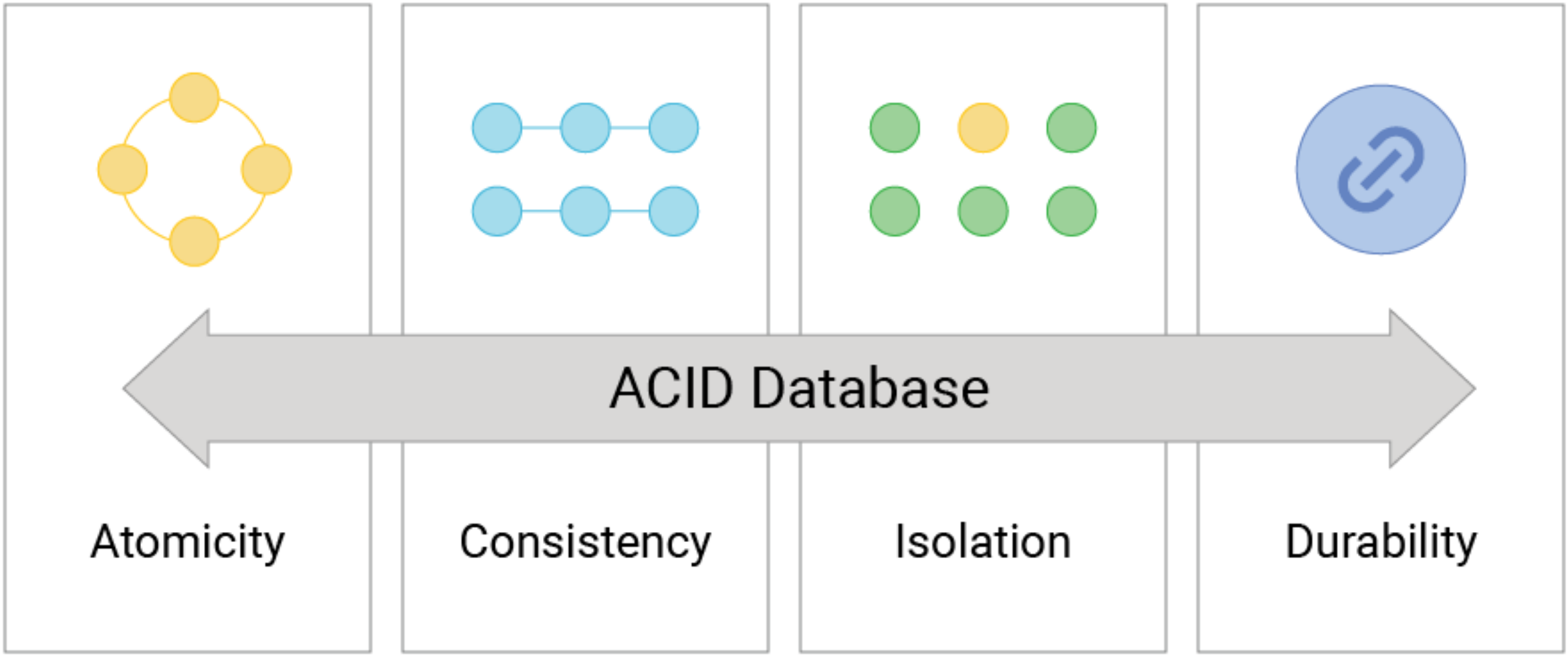
Jeśli w transakcji coś pójdzie nie tak, connection.rollback() cofa wszystkie niezakomitowane operacje. Świetne do atomowych operacji typu "przelew z A do B".

```
try:
    ...; conn.commit()
except: conn.rollback()
```

autocommit · dla ETL / DDL

Jeśli robisz duży batch lub CREATE/DROP, ustaw connection.autocommit = True — każda operacja zapisze się od razu, bez ręcznych commit.

```
conn.autocommit = True
# teraz każdy execute() = natychmiast
```


ACID Database
Atomicity
Consistency
Isolation
Durability

with · zamyka połączenie za Ciebie

Klasyczny błąd juniora: zapomina .close() i serwer Postgres puchnie od "zawieszonych" połączeń. Rozwiązanie: menedżer kontekstu.

```
with psycopg.connect("postgresql://datascience:ds@localhost/datascience") as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM Employee")
        records = cur.fetchall()
        for row in records:
            print(row)

# Po wyjściu z obu 'with':
#   cursor.close() — automatycznie
#   connection.close() — automatycznie
#   BONUS: commit() przy udanym exit, rollback() przy wyjątku
```

Dlaczego to znamy? *Ten sam wzorzec co open() do plików. "with" = cleanup w finally nie Twój problem.*

DEMO · Titanic w PostgreSQL

DEMO

Zapisujemy dane o pasażerach Titanica do Postgresa i odpytujemy:

- Ilu pasażerów przeżyło?
- Jaki był średni wiek ofiar?
- Kto miał największą szansę — klasa 1/2/3?
- Kobiety vs. mężczyźni — kto przeżył częściej?

→ *notebook: Lab4_Databases_MAIN_FILE.ipynb*

```
import pandas as pd
import psycopg
import seaborn as sns

titanic = sns.load_dataset("titanic")

with connection as conn:
    with conn.cursor() as cur:
        cur.execute(create_table)
        for _, row in titanic.iterrows():
            cur.execute(insert, (...))

# Zapytanie analityczne:
cur.execute("""
    SELECT sex, COUNT(*),
           AVG(survived::int)
    FROM Titanic GROUP BY sex
""")
```

Najczęstsze błędy · PostgreSQL

Wracaj do tego slajdu na zadaniach — zaoszczędzisz frustracji.

Zapomniane commit()

Objaw: `INSERT przeszedł, ale w psql tabela pusta`

Fix: conn.commit() PO każdym INSERT/UPDATE/DELETE — lub użyj with

SQL injection / f-string

Objaw: `execute(f"... WHERE name='{name}'")`

Fix: Zawsze execute(sql, (name,)) z %s jako placeholderem

Hasła w kodzie

Objaw: `password='datascience' w repo`

Fix: os.getenv('DB_PASSWORD') + plik .env (w .gitignore)

Postgres 15+ i public schema

Objaw: `permission denied for schema public`

Fix: GRANT ALL ON SCHEMA public TO user — od PG15 trzeba explicit

MongoDB · baza dokumentowa

Zamiast tabel — kolekcje. Zamiast wierszy — dokumenty JSON. Zamiast sztywnego schematu — elastyczność.

Prosty dokument

```
{
  "imie": "Anna",
  "nazwisko": "Kowalska",
  "kierunek": "Informatyka"
}
```

Zagnieżdżony

```
{
  "imie": "Anna",
  "adres": {
    "ulica": "Długa 10",
    "miasto": "Kraków"
  }
}
```

Z listą

```
{
  "imie": "Anna",
  "kursy": [
    "AAP", "WZSI",
    "Python"
  ]
}
```

Każdy dokument w kolekcji może mieć inny zestaw pól — Mongo nie wymusza schematu. Dobre do danych półstrukturalnych (np. logi, produkty w sklepie). Złe, jeśli potrzebujesz transakcji na wielu kolekcjach.

PyMongo · CRUD w 15 liniach

Instalacja: pip install pymongo. Serwer: lokalnie, Docker, lub MongoDB Atlas (darmowy tier 512 MB).

```
from pymongo import MongoClient

# 1. Połączenie — URI lub kwargs
client = MongoClient("mongodb://localhost:27017")
db = client.datascience             # baza (tworzy się sama)
students = db["students"]            # kolekcja (tabela)

# 2. Insert (jeden / wiele)
students.insert_one({"imie": "Anna", "kierunek": "IT"})
students.insert_many([{...}, {...}, {...}])

# 3. Read
for doc in students.find({"kierunek": "IT"}):
    print(doc)

# 4. Update / Delete
students.update_one({"imie": "Anna"}, {"$set": {"kierunek": "Matematyka"}})
students.delete_one({"imie": "Anna"})
```

Filtracja i agregacja · odpowiednik SQL

Operatory filtracji

$gt / $lt większe / mniejsze niż

```
{"price": {"$gt": 100}}
```

$in wartość w liście

```
{"status": {"$in": ["A","B"]}}
```

$and / $or operatory logiczne

```
{"$and": [...]}
```

$regex wyrażenie regularne

```
{"name": {"$regex": "^An"}}
```

dot notation pole zagnieżdżone

```
{"address.city": "Kraków"}
```

Agregacja · odpowiednik GROUP BY

Pipeline: serię etapów przetwarzania dokumentów.

```
# Ile studentów na kierunku?
pipeline = [
  {"$match": {"rok": 2026}},
  {"$group": {
    "_id": "$kierunek",
    "ile": {"$sum": 1}
  }},
  {"$sort": {"ile": -1}}
]

result = list(students.aggregate(pipeline))
```

Bazy specjalistyczne · po co ich tyle?

📈 Szeregi czasowe – Influx / Timescale

IoT + Web 2.0 + observability

Od 2012 eksploduje ilość danych z czujnikiem + timestampem — temperatura co sekundę, ceny akcji co milisekundę, logi serwera co mikrosekundę. Zwykły Postgres się dusi.

Typowe pytania:

- Średnia temp. z ostatniej godziny?
- Peak ruchu sieciowego wczoraj?
- Downsampling: 1 próbka/min → 1/godz

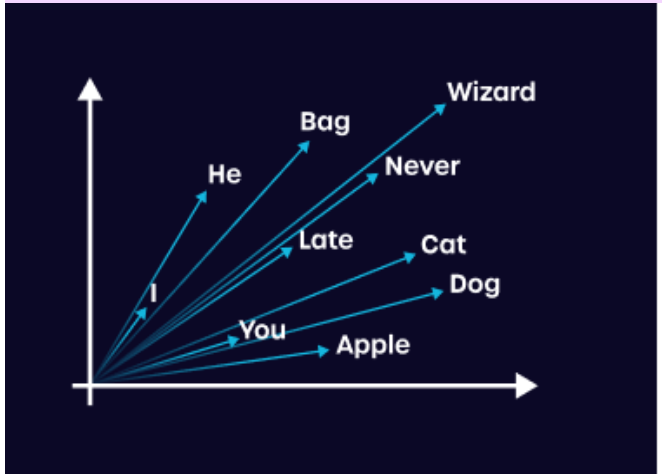
🧠 Bazy wektorowe – PgVector / Qdrant / Weaviate

ChatGPT + RAG + semantic search

Od 2022 (premiera ChatGPT) każdy startup potrzebuje "szukaj po znaczeniu". Embeddingi to wektory 1536-wymiarowe — SQL nie potrafi ich efektywnie porównywać.

Typowe pytania:

- Znajdź 10 podobnych dokumentów
- Jaki produkt przypomina ten obraz?
- RAG: znajdź kontekst dla LLM



I [2, 5]	Bag [5, 8]
You [11, 3]	Wizard [18, 20]
He [3, 8]	Apple [8, 4]
Late [7, 5]	Cat [20, 2]
Never [13, 10]	Dog [23, 1]

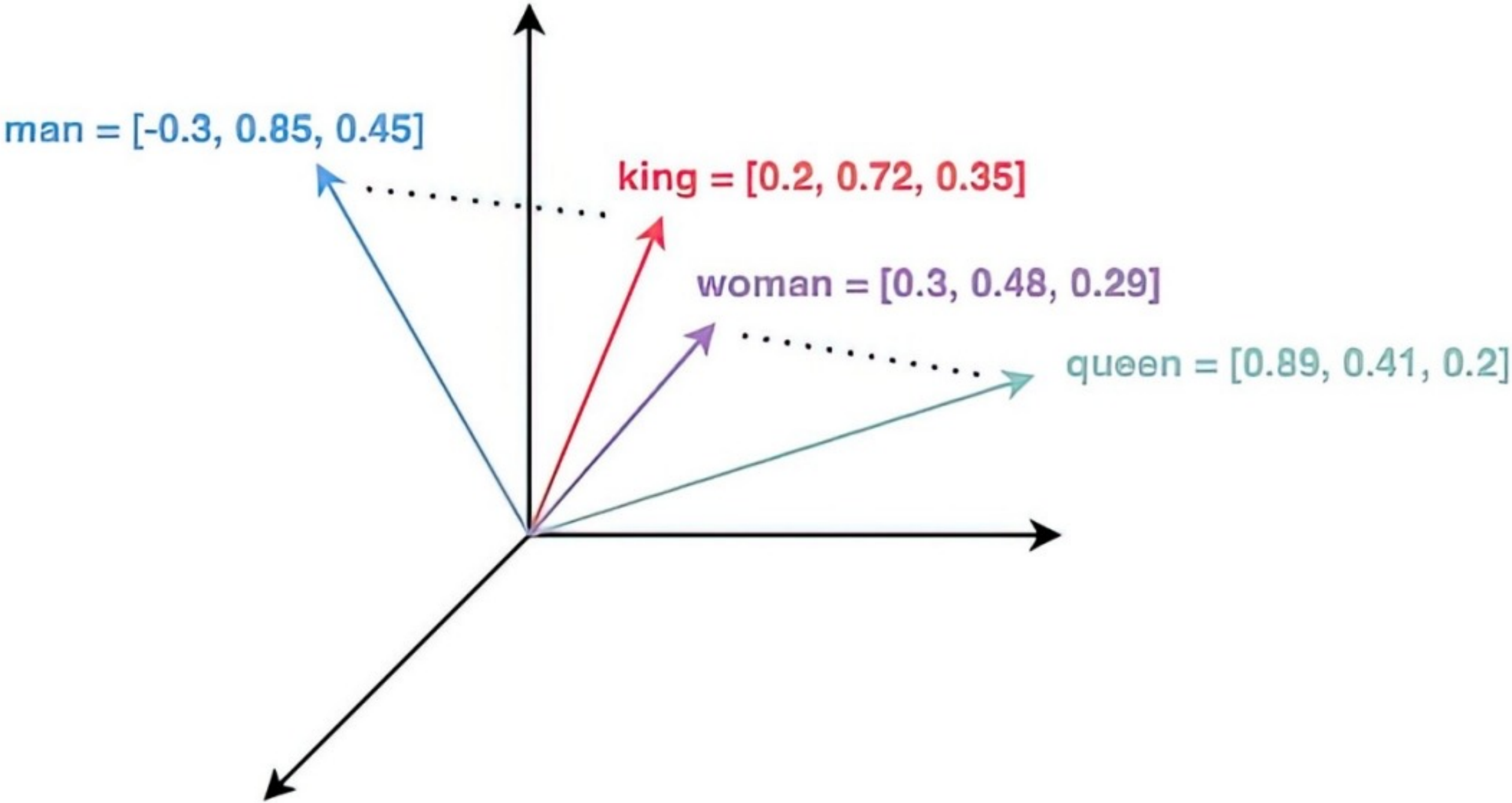
Embeddingi · jak słowa zamieniają się w liczby

Model AI zamienia tekst, obraz lub dźwięk w listę liczb zmiennoprzecinkowych (zwykle 768 lub 1536). Te liczby kodują ZNACZENIE.

"Kot"	→	`[0.21, -0.45, 0.77, ...]`	*1536 wymiarów*
"Kotek"	→	`[0.19, -0.44, 0.81, ...]`	*podobny do "Kot"*
"Samolot"	→	`[0.88, 0.02, -0.31, ...]`	*bardzo różny*

Klucz: podobne znaczenie = bliskie wektory

"Kot" i "Kotek" są blisko. "Kot" i "Samolot" są daleko. Mierzymy to odległością (L2 / Euklides) lub kosinusem kąta między wektorami. Baza wektorowa to system, który umie znaleźć K NAJBLIŻSZYCH wektorów w milisekundach — nawet przy miliardach punktów. Algorytm: HNSW (hierarchiczne grafy) lub IVFFlat.


man = [-0.3, 0.85, 0.45]
king = [0.2, 0.72, 0.35]
woman = [0.3, 0.48, 0.29]
queen = [0.89, 0.41, 0.2]

Bazy wektorowe · kiedy co wybrać

Od 2022 powstało ich kilkanaście. W 2026 stabilizuje się obraz: dwie kategorie — rozszerzenia do istniejących baz vs. dedykowane silniki.

pgvector

rozszerzenie

✓ Wybierz gdy:

Masz już Postgresa. Do 10–50 mln wektorów. AI feature do istniejącej aplikacji.

✗ Pomiń gdy:

Więcej niż 100 mln, ekstremalne QPS.

Qdrant

dedykowany · open-source

✓ Wybierz gdy:

Self-hosting, najlepsza latencja (~4ms p50). Rust. Pełna kontrola.

✗ Pomiń gdy:

Trzeba utrzymywać osobny serwis.

Pinecone

managed SaaS

✓ Wybierz gdy:

Zero ops. Klikasz i działa. Prototypy, startupy. Auto-skalowanie.

✗ Pomiń gdy:

Drogie przy dużej skali. Vendor lock-in.

Weaviate

dedykowany + hybrid

✓ Wybierz gdy:

Wbudowany hybrid search (vector + BM25). Multi-tenancy. Enterprise.

✗ Pomiń gdy:

Stromsza krzywa nauki, Java-based.

pgvector · Postgres zostaje Postgresem, ale wie, co to znaczenie

Rozszerzenie PostgreSQL dodające typ VECTOR i operatory odległości. Instalacja: CREATE EXTENSION vector;

1. Tabela z kolumną wektorową (SQL)

```
CREATE EXTENSION vector;
CREATE TABLE documents (id SERIAL, content TEXT, embedding VECTOR(1536));
CREATE INDEX ON documents USING hnsw (embedding vector_l2_ops);
```

2. Wstaw embeddingi (Python)

```
from openai import OpenAI
embedding = OpenAI().embeddings.create(input=text, model="text-embedding-3-small").data[0].embedding
cur.execute("INSERT INTO documents (content, embedding) VALUES (%s, %s)", (text, embedding))
```

3. Semantic search (SQL!)

```
SELECT content FROM documents
ORDER BY embedding <-> %s   -- <-> = dystans L2, <=> = cosine, <#> = inner product
LIMIT 5;
```

Czego dziś NIE zdążyliśmy

To dopiero początek — w prawdziwym projekcie spotkasz jeszcze:

SQLAlchemy / ORM

warstwa abstrakcji

ORM mapuje klasy Python ↔ tabele SQL. Django ORM to to samo.

Migracje (Alembic)

zarządzanie schematem

Jak Git dla bazy — wersjonowanie zmian schematu w zespole.

Connection pooling

wydajność

pgbouncer, SQLAlchemy pool — nie otwieraj 1000 połączeń.

Replikacja & backup

produkcja

master-slave, streaming replication, pg_dump. Ratuje życie.

Redis

cache + kolejki

In-memory key-value. Session store, rate limit, pub/sub.

RAG pipeline

AI

LangChain + pgvector + LLM. Prawdziwe zastosowanie wektorów.

Zapamiętaj te hasła. Wygoogluj w domu — to mapa Twojej dalszej nauki.

Zadania

Zadanie 1 · PostgreSQL

Korzystając z Random User API (`https://randomuser.me/api/?results=30`), wygeneruj 30 użytkowników i zapisz ich (imię, nazwisko, email, adres, wiek, płeć) w tabeli PostgreSQL. Sprawdź SQL-em: ile mężczyzn, średni wiek, w ilu krajach mieszkają.

💡 *Tip: requests.get(...).json()["results"] zwróci listę dictów — iteruj i INSERT-uj po jednym.*

Zadanie 2 · MongoDB

Z API GeckoTerminal (`https://api.geckoterminal.com/api/v2/networks`) pobierz informacje o dostępnych sieciach kryptowalutowych. Zapisz je jako dokumenty w kolekcji MongoDB. Użyj agregacji, żeby policzyć liczbę sieci per typ.

💡 *Tip: MongoDB akceptuje dict bezpośrednio — żadnej parametryzacji jak w SQL.*

Zadanie 3 · BONUS · pgvector

Dla chętnych: stwórz tabelę z 5 opisami filmów i ich embeddingami (mogą być losowe VECTOR(3) dla uproszczenia). Napisz zapytanie, które znajdzie 3 najbardziej podobne do podanego wektora zapytania.

Zapisz: zadanie_1.ipynb · zadanie_2.ipynb · zadanie_3_bonus.ipynb (opcjonalne) · Czas: ~15 min

Utknąłeś? Zerknij na slajd "Najczęstsze błędy" — odpowiedź pewnie tam jest.

Sprawdź, co umiesz

Pomyśl 30 sekund nad każdym pytaniem. Nie odpowiadaj na głos — to ma być Twoja głowa, nie cudza.

1. Czym różni się baza relacyjna od dokumentowej? Daj po jednym przykładzie.

2. Co robi connection.commit() i co się stanie, jeśli zapomnisz?

3. Dlaczego execute(sql, (name,)) jest lepszy niż f-string z nazwą?

4. Kiedy w 2026 wybierzesz pgvector, a kiedy Qdrant?

5. Po co istnieją bazy szeregów czasowych, skoro mamy Postgresa?

6. Co to jest embedding i dlaczego "kot" i "kotek" są bliżej siebie niż "kot" i "samolot"?

Nie umiesz na któreś? To nie wstyd — wróć do odpowiedniego slajdu PRZED następnymi zajęciami.

Dziękuję za uwagę

PYTANIA?